Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Департамент цифровых, робототехнических систем и электроники
Институт перспективной инженерии

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины «Патерны проектирования»
Тема: Структурные паттерны: адаптер, мост

Выполнил:
Чекалин Егор Юрьевич
4 курс, группа ПИЖ-б-о-23-2
09.03.04 «Программная инженерия»,
очная форма обучения

_____
(подпись)

Работу проверил:
Николаев Е.И.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

1 Цель работы

Реализовать решение задачи из предметной области с использованием структурных паттернов «Адаптер» и «Мост». Предметная область: приложение бронирования авиабилетов aviacassa (Java, Spring Boot).

2 Краткие теоретические сведения

Адаптер (Adapter) позволяет объектам с несовместимыми интерфейсами работать вместе: адаптер реализует требуемый интерфейс и внутри себя переадресует вызовы методам существующего класса, переводя данные из одного формата в другой.

Мост (Bridge) разделяет одну большую систему на два независимых дерева классов: абстракцию (что делается) и реализацию (как делается). Абстракция хранит ссылку на интерфейс реализации, и обе иерархии могут изменяться независимо, а конкретная реализация подставляется во время выполнения.

3 Ход работы

3.1 Анализ кода приложения

Паттерн «Адаптер» применяется в интеграции со Spring Security: класс CustomUserDetailsService реализует интерфейс UserDetailsService и адаптирует JPA-сущность User к типу UserDetails, переводя вызов loadUserByUsername() в обращение к репозиторию.

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService
        implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException(
                        "Пользователь не найден: " + username));

        return new org.springframework.security.core.userdetails.User(
                user.getUsername(),
                user.getPassword(),
                Collections.singletonList(
                        new SimpleGrantedAuthority(user.getRole().name()))
        );
    }
}
```

Адаптер применяется и при интеграции с внешней платежной системой. Внешний шлюз возвращает идентификатор транзакции в собственном формате (строка EXT-…), а внутреннему коду нужен объект PaymentResult. Адаптер реализует интерфейс PaymentGateway и скрывает протокол внешней системы.

Паттерн «Мост» применяется в подсистеме уведомлений. Тип уведомления (о бронировании, о платеже) независим от канала доставки (email, SMS): абстракция Notification хранит ссылку на интерфейс MessageSender, уведомления и отправители подставляются независимо.

3.2 Демонстрационная программа

Для демонстрации написана программа на Java. Адаптер ExternalPaymentApiAdapter оборачивает внешнюю платежную систему и приводит ее ответы к интерфейсу PaymentGateway. Мост составлен из двух иерархий: уведомления и отправители; связь задается конструктором.

```
class ExternalPaymentApiAdapter implements PaymentGateway {
    private final ExternalPaymentApi api;

    public ExternalPaymentApiAdapter(ExternalPaymentApi api) {
        this.api = api;
    }

    @Override
    public PaymentResult pay(String cardNumber, BigDecimal amount) {
        String raw = api.makeTransaction(amount.doubleValue(),
cardNumber);
        return new PaymentResult(raw.startsWith("EXT-"), raw);
    }
}

abstract class Notification {
    protected final MessageSender sender;

    protected Notification(MessageSender sender) {
        this.sender = sender;
    }

    public void send(String recipient) {
        String answer = sender.deliver(recipient, buildText());
        System.out.println("  отправлено: " + answer);
    }

    protected abstract String buildText();
}
```

Диаграмма классов демонстрационной программы приведена на рисунке 1.

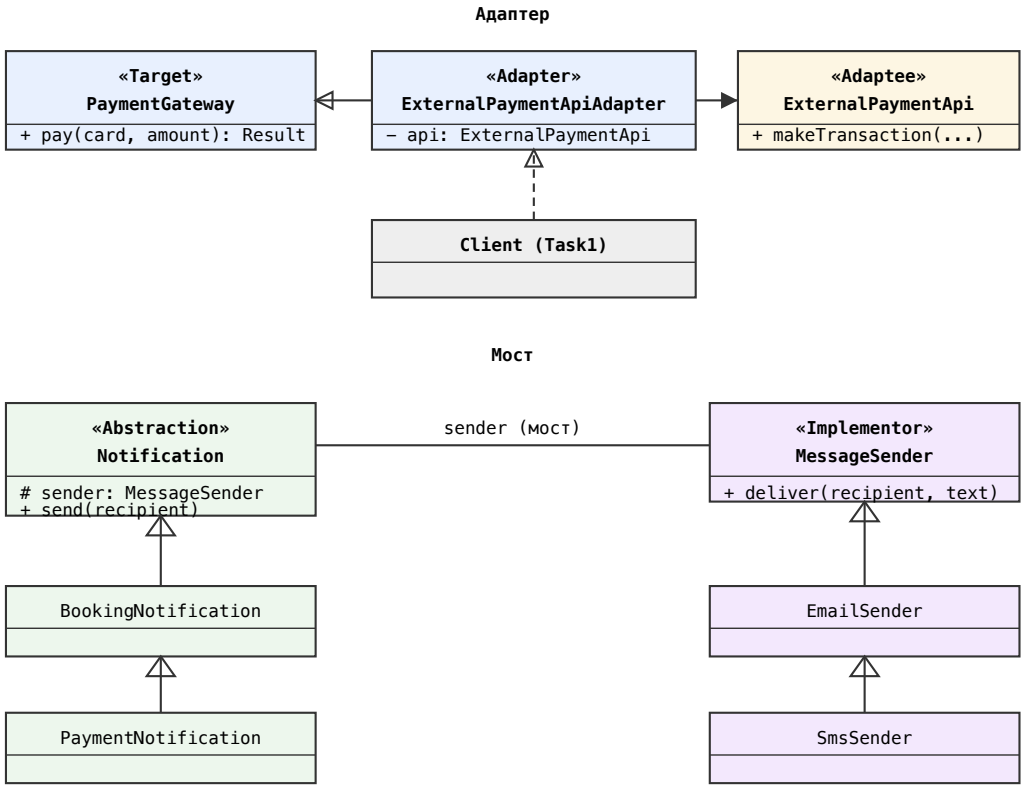


Рисунок 1 - Диаграмма классов паттернов «Адаптер» и «Мост»

В точке входа программы клиент работает с внешней платежной системой только через адаптер, а уведомления собираются из любого сочетания типа и канала доставки:

```
public static void main(String[] args) {
    System.out.println("Адаптер:");
    PaymentGateway gateway =
            new ExternalPaymentApiAdapter(new ExternalPaymentApi());
    PaymentResult result =
            gateway.pay("4111111111111111", new BigDecimal("15800.00"));
    System.out.println("  результат: " + result);

    System.out.println();
    System.out.println("Мост:");
    Notification bookingEmail = new BookingNotification(new
EmailSender());
    Notification paymentSms = new PaymentNotification(new SmsSender());
    bookingEmail.send("user@example.com");
    paymentSms.send("+79001234567");
}
```

Результат запуска программы показан на рисунке 2.

```
Адаптер:
  результат: PaymentResult[success=true, transactionId=EXT-1111:1580000]

Мост:
  отправлено: email на user@example.com: Бронирование PENDING, оплата в течение 15 минут
  отправлено: SMS на +79001234567: Платеж успешно завершен, билеты доступны в личном кабинете
```

Рисунок 2 - Результат выполнения программы

Результат: адаптер переадресовал вызов pay() внешнему API и вернул PaymentResult с success=true (EXT-1111:1580000). Мост собрал уведомления из разных сочетаний иерархий: бронирование по email, платеж по SMS.

4 Вывод

Изучены структурные паттерны «Адаптер» и «Мост». В aviacassa адаптер применяется в интеграции со Spring Security (CustomUserDetailsService), мост в подсистеме уведомлений отделяет тип уведомления от канала доставки. Работа паттернов подтверждена демонстрационной программой.